

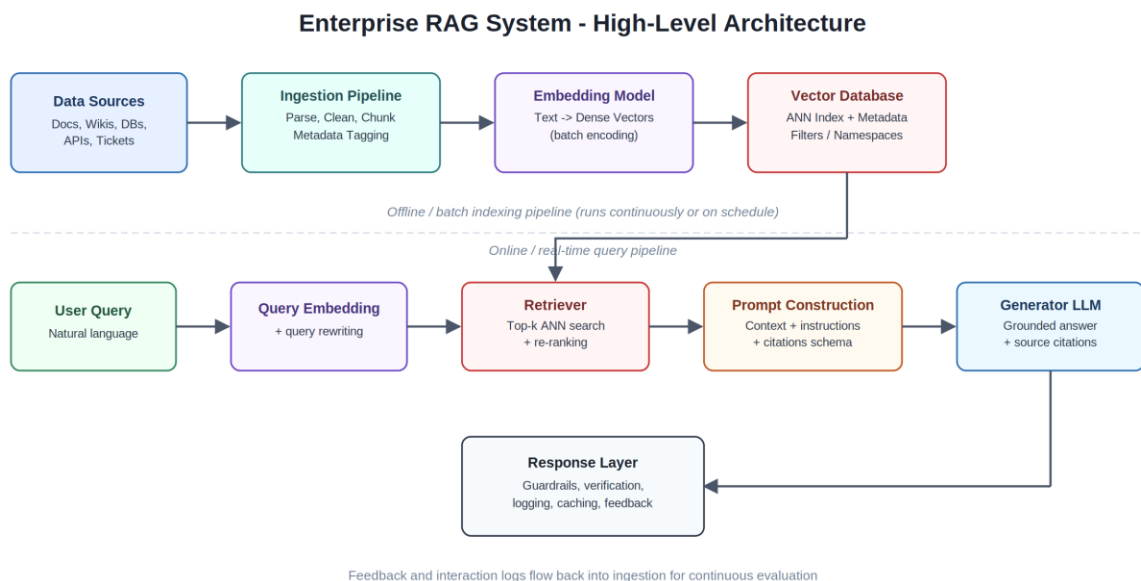
Designing Enterprise Retrieval-Augmented Generation Systems

Technical Research Report

Retrieval-Augmented Generation (RAG) combines a large language model (LLM) with an external knowledge retrieval layer, allowing enterprise applications to generate answers grounded in proprietary, up-to-date data rather than relying solely on the model's parametric memory. This report outlines the core architectural components required to design a production-grade RAG system and the design decisions that determine its accuracy, latency, and reliability.

1. Architecture Overview

An enterprise RAG system is generally split into two pipelines: an offline ingestion pipeline that indexes organizational content, and an online query pipeline that retrieves relevant context and generates a grounded response at request time. Figure 1 summarizes the end-to-end flow, from raw data sources through embedding, storage, retrieval, prompt assembly, and generation, with feedback signals looped back for continuous improvement.



2. Embeddings

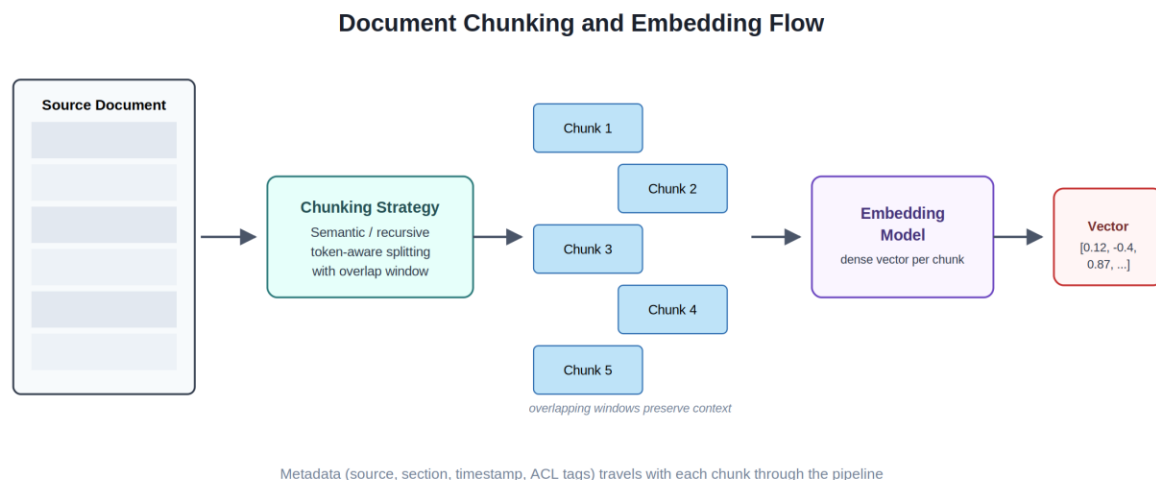
Embeddings convert text into dense numerical vectors positioned so that semantically similar content lies close together in vector space. Model choice directly affects retrieval quality and cost.

- **Model selection:** domain-tuned or multilingual embedding models (e.g., E5, BGE, OpenAI/Voyage embeddings) generally outperform general-purpose models on enterprise jargon and non-English content.
- **Dimensionality trade-off:** higher-dimensional embeddings capture more nuance but increase storage and search latency; 384-1024 dimensions is a common production range.
- **Consistency:** the same embedding model (and version) must be used for both indexing and querying, since vectors from different models are not comparable.

- **Asymmetric embeddings:** some models offer separate query and passage encoders, which improves match quality for short queries against long documents.

3. Chunking

Because embedding models and LLM context windows have fixed capacity, source documents must be split into chunks before indexing. Chunking strategy is one of the highest-leverage decisions in RAG design, since retrieval can only be as good as the unit being retrieved.



- **Fixed-size chunking:** simple and fast, but risks splitting sentences or ideas mid-thought.
- **Recursive/structure-aware chunking:** splits along headings, paragraphs, or sections first, falling back to token limits only when necessary.
- **Semantic chunking:** groups sentences by embedding similarity so each chunk represents one coherent idea.
- **Overlap:** a 10-20% overlap between adjacent chunks reduces the risk of losing context at chunk boundaries.

4. Retrieval

At query time, the retriever converts the user's question into an embedding and searches the vector index for the most relevant chunks, typically returning the top-k candidates for further processing.

- **Dense retrieval:** approximate nearest neighbor (ANN) search over embeddings, effective for semantic/paraphrased queries.
- **Hybrid retrieval:** combining dense vectors with sparse keyword search (BM25) improves recall for exact terms, product codes, or acronyms that embeddings alone may miss.
- **Re-ranking:** a cross-encoder re-ranker scores the initial candidate set more precisely before final selection, improving precision at a modest latency cost.
- **Metadata filtering:** restricting search by document type, date, department, or access permissions keeps results relevant and enforces data governance.

5. Prompt Construction

Retrieved chunks are assembled into a prompt alongside the user query and system instructions. Careful prompt design determines whether the model treats retrieved context as authoritative.

- **Context ordering:** placing the most relevant chunks closest to the question reduces the effect of position bias in long contexts.
- **Grounding instructions:** explicit instructions to answer only from provided context, and to state uncertainty when evidence is insufficient, reduce fabrication.
- **Citation formatting:** requiring the model to tag claims with source identifiers enables traceability and downstream verification.
- **Context budget management:** truncation or summarization strategies are needed when retrieved content exceeds the model's context window.

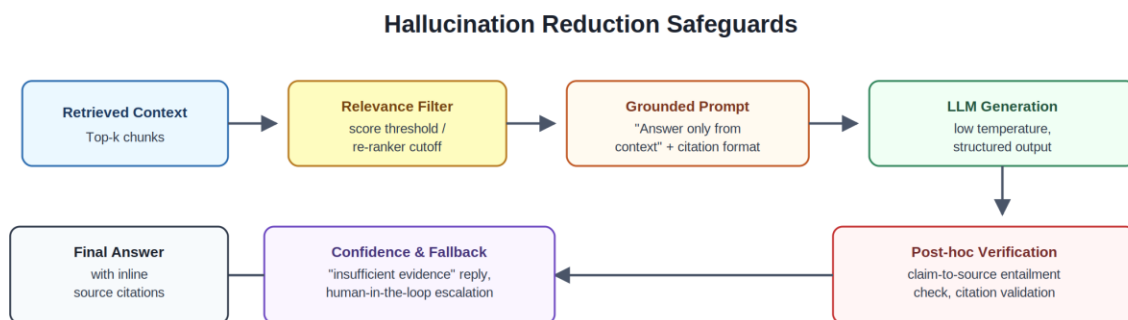
6. Vector Databases

The vector database stores embeddings alongside metadata and serves low-latency similarity search at scale. Selection depends on data volume, update frequency, and existing infrastructure.

- **Index type:** HNSW graphs offer strong speed/accuracy trade-offs; IVF and product quantization reduce memory footprint for very large collections.
- **Managed vs. self-hosted:** managed services (e.g., Pinecone, Weaviate Cloud, Zilliz) reduce operational overhead; self-hosted options (e.g., Milvus, Qdrant, pgvector) offer more control over data residency and cost.
- **Freshness:** enterprise systems need incremental upsert and delete support so the index reflects document changes without full re-indexing.
- **Multi-tenancy and access control:** namespaces or metadata-based filters are required to isolate data across teams, customers, or permission levels.

7. Hallucination Reduction

Even with retrieval, LLMs can generate claims not supported by the retrieved context. A layered set of safeguards reduces this risk, illustrated in Figure 3.



Each stage narrows the model's freedom to invent facts outside retrieved evidence

- **Relevance filtering:** discarding low-similarity-score chunks before they reach the prompt prevents the model from working with noise.
- **Low-temperature, structured generation:** reduces creative deviation and encourages consistent, verifiable output.
- **Post-hoc verification:** an entailment or fact-checking pass compares generated claims against retrieved evidence before the answer is returned.
- **Abstention behavior:** the system should explicitly answer "insufficient evidence" rather than guess, with an option to escalate to a human reviewer.

8. Future Improvements

- **Agentic retrieval:** multi-step, self-querying retrieval where the model plans sub-queries and iteratively refines context before answering.
- **Graph-augmented RAG:** combining vector search with knowledge graphs to capture relationships that pure similarity search misses.
- **Adaptive chunking:** learned chunk-boundary models that adjust granularity per document type rather than fixed heuristics.
- **Continuous evaluation:** automated regression testing of retrieval and answer quality as the underlying corpus and models evolve.
- **On-the-fly personalization:** blending user or role context into retrieval ranking without compromising data isolation.

Conclusion: A well-designed enterprise RAG system treats retrieval quality, prompt grounding, and verification as equally important to model choice. Investing in chunking strategy, hybrid retrieval, and layered hallucination safeguards typically yields larger accuracy gains than upgrading the generator model alone.